MSSE 601 – SOFTWARE ENGINEERING FUNDAMENTALS

WEEK 8 Completed Software Project Plan

Golden Bike Shop Project

Group Category: SE Fundamentals

Group Name: SEF 1 -NCM Innovators

Prepared by:

Minnette Johnson

Nasar Roble

Carolann Mora

Peter Olayiwola

Regis University

Submitted for: MSSE 601 – Software Engineering Fundamentals

Date: October 18, 2026

Table Of Content

Introduction **4**
1. Project Charter and Process Selection **4**
1.1 The problem 4
1.2 Stakeholders 4
1.3 Scope 5
1.4 Assumptions, constraints and risks 5
1.5 Process selection 6
2. Requirements Specification **6**
2.1 Functional requirements 6
2.2 Non-functional requirements, in priority order 8
3. User Journeys and Interface Design **9**
3.1 Key user journeys 9
3.2 Screens 10
3.3 Design reasoning 10
3.4 Accessibility (WCAG 2.1 AA) 10
4. Architecture Description **11**
4.1 Approach 11
4.2 Data authority and caching 11
4.3 Services 11
4.4 How the architecture meets the requirements 12
4.5 The case against this architecture 13
5. Data Design **13**
5.1 How the data is stored 13
5.2 Entities 14
5.3 Integrity rules 15
5.4 Status lists (new, row 1) 15
5.5 How data moves 15
5.6 Checking the model against real cases 16
6. Quality and Test Plan **16**
6.1 Quality processes 16
6.2 Testing inside our process 17
6.3 Key tests 17
6.4 Where we stop, and the risk we accept 18
6.5 When a defect escapes 19
Part 2: Where the Components Disagreed, and What We Did **19**
Part 3: Deployment and Maintenance Outlook **23**
How the system would be released 23

Rollback plan..... 23
What will be hardest to maintain, and why..... 24
Conclusion: What We Would Build First, and Why..... 24

Introduction

The Golden Bike Shop Management System is one web-based platform that connects Golden Bike Shop's four stores. It replaces each store's separate software and spreadsheets with a single source of truth for retail inventory and sales, the 400-bike rental fleet (including e-bikes that report battery and location), and the shared repair workshop.

It is for the people who run the shop and the people who use it:

- **Store Clerks** check and update stock across all four stores.
- **Rental Agents** check availability and book bikes, and only see e-bikes that are charged and reported.
- **Mechanics** schedule workshop bays, track repairs and allocate parts.
- **Management** sees consolidated four-store sales.
- **Customers** book rentals and check repair status online.

The package brings together six components produced between Weeks 2 and 7, in the order a reader needs them: why the system exists and how we work (Section 1), what it must do (2), how people use it (3), how it is built (4), what it stores (5) and how we know it works (6). Each section reflects the reconciliation changes listed in Part 2, where we record every place the components disagreed and which document changed. Part 3 sets out how the system would be released and maintained, and the conclusion states what we would build first.

Per-week Decision Records, AI Use Records and ownership tables are collected in the separate Part 5 file.

1. Project Charter and Process Selection

1.1 The problem

Golden Bike Shop sells bikes, parts and accessories from four stores, rents a 400-bike fleet that includes e-bikes, and runs a shared repair workshop. Each store uses its own software and spreadsheets, which causes three recurring failures:

- **Inventory blind spots.** Staff cannot see stock at other stores, which loses sales and delays orders.
- **Workshop delays.** Mechanics cannot see parts at other stores.
- **Rental errors.** Manual tracking leads to double bookings and bikes lost from availability.

If nothing changes, these failures continue to cost revenue and customer trust, and competitors with online booking take market share.

1.2 Stakeholders

Group	Who	What they need
Deciders	Business owners, executive management	Efficiency, growth, cost control, consolidated reporting
Payers	Business owners, investors	Return on investment, predictable maintenance cost, staying within the $500,000 budget
Daily users	Store Clerks, Rental Agents, Mechanics	A clear interface, fast checkout, live inventory, clear bay scheduling
Affected	Customers, parts suppliers	Easy online booking and repair status; better demand forecasts

Two tensions shaped the design. Management wants low build and maintenance cost, while staff want features that match their current habits. Customers want fast turnaround, while mechanics need full intake checks and e-bike battery checks before a bike goes out.

1.3 Scope

In scope: a shared multi-store database for inventory and sales; rental fleet booking; e-bike telemetry, so only charged and reporting e-bikes are rented; workshop bay scheduling, job tracking and parts allocation; staff and customer web portals with role-based access; and rental deposits and charges through an external payment gateway (added in Part 2, row 4).

Out of scope: payroll, tax and accounting (data exports only); marketing automation and a standalone CRM; telemetry hardware; and low-level server and pipeline management, which a managed cloud platform handles.

1.4 Assumptions, constraints and risks

We assume a $500,000 implementation budget, a fixed eight-week course schedule, and a four-person team working asynchronously on Slack with a weekly Wednesday sync.

Risk	Impact	Mitigation
Telemetry latency	An undercharged e-bike could be rented	Any e-bike with telemetry older than 15 minutes is unavailable
Different workflows at each store	Adoption friction and user error	Standardise intake and rental workflows before final interface design
Seasonal and event peaks	Booking slowdowns	Managed cloud platform with auto-scaling; peak load recorded as an accepted risk (Part 2, row 12)

Fixed schedule and team size	Limited bandwidth	Weekly scope control and a 24-hour internal review buffer

1.5 Process selection

We chose an **iterative Agile approach with lightweight Scrum practices**: one-week sprints aligned to course milestones, asynchronous check-ins on Slack and a weekly synchronous meeting. Deliverable ownership rotates every week.

We chose it for three reasons. Rules for fleet availability, telemetry thresholds and bay scheduling needed refining as the design progressed. Workshop schedules affect rental availability, so cross-area dependencies had to be checked often. And an eight-week schedule needs early, frequent feedback.

We rejected two alternatives:

- **Waterfall** locks requirements upfront, risking costly rework when gaps surface late. We gave up on its predictable phases.
- **V-Model** suits safety-critical systems and adds overhead for a web application. We kept its one useful idea, that every requirement has a matching test, through the traceability in Section 6.

We would revisit the choice if regulation imposed fixed, signed-off requirements (switch to Waterfall), or if the team became full-time with overlapping hours (switch to full Scrum).

2. Requirements Specification

The system has 15 functional requirements and 8 non-functional requirements. Requirements marked *(changed)* or *(new)* were revised during reconciliation; the Part 2 row is given for each. These functions operate as one connected system: a bike in the workshop is never offered for rental, and mechanics see parts at all four stores.

2.1 Functional requirements

ID	User story	Acceptance criteria
FR-01	As a Store Clerk, I can view product and part inventory across all four stores.	Search by item and store; each result shows item, quantity and store from the central database.
FR-02	As a Store Clerk, I can update inventory quantities when items are received or sold.	An authorized clerk can raise or lower a quantity; other users see the new quantity.

FR-03	As a Rental Agent, I can check which bikes are available for a requested period.	Bikes that are reserved, rented, In Service, or e-bikes failing FR-06 are not shown as available.
FR-04 *(changed, row 3)*	As a Rental Agent, I can create a reservation for **one or more**available bikes.	Records the customer, each bike and the period. A bike whose period conflicts with another booking or an unavailable status is rejected.
FR-05 *(changed, row 3)*	As a Customer, I can reserve **one or more** bikes through the customer portal.	Same rules as FR-04, stored in the same central reservation records.
FR-06 *(changed, rows 2 and 13)*	As a Rental Agent, I can see current e-bike battery level and fault codes.	Telemetry older than 15 minutes, battery below 30%, or an active fault code makes the e-bike unavailable until a valid reading arrives.
FR-07	As a Mechanic, I can schedule a bike for workshop service.	Records the bike, the time and the bay. The same bay cannot be given to overlapping active jobs. Scheduling a rental bike sets it to In Service.
FR-08 *(changed, row 1)*	As a Mechanic, I can update a repair job's status.	Uses the repair-job status list in Section 5.4; staff and the customer see the current status.
FR-09	As a Mechanic, I can allocate parts to a repair job.	Shows part availability at all four stores; allocated parts are recorded against the job and inventory is reduced.
FR-10	As Management, I can view consolidated sales for all four stores.	Shows sales by store and period from central records.
FR-11	As a Customer, I can view the status of my own repair.	Shows the current status; customers can see only their own repairs.
FR-12	As a Business Owner, I want access based on user roles.	Clerks, Rental Agents, Mechanics, Management and Customers reach only the functions for their role.
FR-13 *(new, row 4)*	As a Rental Agent, I can take a deposit, final charge or refund through the payment gateway.	Card details stay with the gateway; the system stores only the gateway reference, type and amount.

FR-14 *(new, row 16)*	As Management, I can set rental rates by bike type and period.	Rates are hourly, daily or weekly with effective dates; a booking keeps the rate agreed when it was made.
FR-15 *(new, row 16)*	As a Customer, I receive booking confirmations and repair-ready notices.	Sent by email or SMS only after the related change is saved.

2.2 Non-functional requirements, in priority order

Rank	Requirement	Measurable criterion
1	NFR-01 Cross-store data consistency	100% of consistency tests return the same committed value for the same record at all four stores.
2	NFR-02 Concurrent transaction integrity	Across 100 simulated conflicting booking and bay scenarios, zero double bookings.
3	NFR-03 E-bike telemetry freshness	100% of e-bikes with telemetry older than 15 minutes are excluded from availability.
4	NFR-04 Role-based access security	100% of unauthorized access attempts are denied; customers see only their own records.
5	NFR-08 Data encryption *(changed, row 4)*	Telemetry, customer licence numbers and payment references are encrypted in transit and at rest.
6	NFR-06 Availability and recovery *(changed, row 11)*	At least 95% weekly uptime; core functions restored within 30 minutes; no more than 5 minutes of committed data lost.
7	NFR-05 Performance	Searches within 3 s and submissions within 5 s, for at least 95% of requests.
8	NFR-07 Usability	At least 9 of 10 representative users finish their main task unaided within 3 minutes.

We ranked by dependency, risk and business value. Consistency comes first because every other function relies on correct central data. Concurrency comes next because a failure double-books bikes or bays. Telemetry freshness follows because the charter names it as a risk. Encryption ranks above availability because exposing data is worse than a short outage. Availability ranks above performance because a slow system still works and a down one does not. Status information never relies on colour alone (Section 3.4).

3. User Journeys and Interface Design

Three journeys cover the rental flow, which earns the most revenue and carries the most risk: finding a bike, renting it, and taking it back. Personas now match the Section 2 roles (Part 2, row 6), and bike status labels use the list in Section 5.4 (row 1).

3.1 Key user journeys

Journey 1: Find and check a bike (Jake, Rental Agent; Screens 1 and 2)

1. Jake filters the Fleet table by status (Available) and store (Golden).
2. He opens a bike's details to check its condition and battery, then adds a note and saves.

The system shows matching rows, opens the bike's current record, then validates and saves the note. Battery level and fault codes come from telemetry and are read-only.

If it fails: if another agent books the bike at the same moment, the table refreshes and shows it as Reserved. If a required field is blank, saving is blocked and the field is highlighted.

Journey 2: Rent one or more bikes (Jake, Rental Agent; Screen 3)

1. Jake selects one or more available bikes and opens New Rental.
2. He enters the customer, start time and expected return, and takes the deposit (FR-13).
3. He confirms the rental.

The system pre-fills the selected bikes and checks required fields. On confirmation, each bike changes from Available to Rented and a receipt is generated.

If it fails: a missing name or return date blocks confirmation with "Rental cannot be completed. Please enter the customer's name and return date." A declined deposit leaves the bikes Available. *(The smart-lock unlock step was removed, because telemetry hardware is out of scope; Part 2, row 2.)*

Journey 3: Return and inspect (Elena, Mechanic; Screen 4)

1. Elena scans the returned bike and reviews its condition and telemetry fault codes.
2. She chooses the bike's next status, Available or In Service, and confirms.

The system calculates the rental duration and any late charge, takes the balance through the gateway, and sets the chosen status. In Service removes the bike from rental searches.

If it fails: for a late return, the overage is calculated and confirmed before billing. If Elena chooses Available while fault codes are active, a confirmation step forces her to verify the bike first.

3.2 Screens

Screen	Supports	Main contents
1. Fleet *(renamed from Inventory Management, row 7)*	Journey 1	Table of bike, type, status, location and actions, with search, filters and Add Bike
2. Bike details	Journey 1	Bike ID, type, status, location, condition, read-only battery and fault codes, Save and Cancel
3. New rental *(changed, row 3)*	Journey 2	Customer details, one or more selected bikes, start and expected return, deposit, Confirm and Cancel
4. Bike return	Journey 3	Rental and bike details, return time, condition, a required Available or In Service choice, Confirm

Not yet designed (Part 2, row 8): the parts-inventory screen (FR-01/02), customer booking (FR-05), customer repair status (FR-11), workshop scheduling and parts allocation (FR-07/09), and sales reporting (FR-10). Customer booking and repair status come first.

3.3 Design reasoning

The design prioritizes visibility, consistency, recognition over recall, and error prevention.

Decision	Options	Choice and reason	Trade-off
Fleet layout	Cards or a table	A table, because agents compare several bikes at once	A denser screen
Editing	Inline in the table, or a separate screen	A separate details screen, which reduces edits to the wrong bike	One extra step
Return status	Mark every return Available automatically, or require a choice	Require a choice, because a returned bike may be damaged	A slightly longer return
Status display	Colour alone, or colour plus text	Colour plus text labels	Slightly more space

3.4 Accessibility (WCAG 2.1 AA)

- **Colour:** status is always written as text beside the coloured dot.

- **Keyboard:** controls follow a logical tab order with a visible focus indicator, including the custom Available / In Service cards on Screen 4.
- **Text and contrast:** labels sit above fields, and the dark navy text and buttons meet contrast ratios.
- **Errors:** messages say what went wrong and what to do next.

4. Architecture Description

4.1 Approach

The core is kept simple and transactional. Telemetry is isolated because it has a different workload and failure profile, and further services are split out only when a concrete need appears. The result is a hybrid:

- **Presentation:** a staff portal (Clerks, Rental Agents, Mechanics, Management) and a customer portal. Neither touches the database directly.
- **Core:** one Spring Boot modular monolith containing a Backend-for-Frontend (BFF) / API layer, a Retail module, a Service Booking module and an Identity module.
- **Telemetry service:** deployed independently. It receives readings, checks freshness and enforces the 15-minute rule.
- **Infrastructure:** PostgreSQL as the system of record, Redis as a read cache, and a durable message broker for telemetry events, Kafka (Part 2, row 5).

Retail and Service Booking share one deployment because their workflows need strong consistency and shared concurrency controls. Module boundaries are enforced in code (separate `retail`, `booking` and `identity` packages), so a module can later be extracted as a service.

4.2 Data authority and caching

- **Writes go to the database first.** The application validates the request, commits it to PostgreSQL in a transaction, and only then clears the affected cache entries.
- **Reads check the cache first.** On a cache miss, the application reads PostgreSQL (or the telemetry snapshot) and stores the result in the cache with a time limit.
- **The cache is never the source of truth.** If Redis fails, reads fall back to the database. Searches get slower but stay correct.

4.3 Services

Service	Responsible for	Triggered by	Supports

BFF / API layer	Authentication hand-off, role checks, validation, one response per screen	User action	FR-12, NFR-04
Identity and access	Users, roles, permissions, sessions; customers see only their own repairs	User action	FR-11, FR-12
Inventory and sales	Per-store stock, catalogue, sales, parts used in repairs	User action	FR-01, 02, 09, 10
Rental reservation	Availability, bookings, rental agreements, returns, damage, overage, payment	User action	FR-03 to 05, FR-13, FR-14, NFR-02
Workshop scheduling	Repair jobs, bay and mechanic scheduling, status, parts; sets bikes In Service	User action	FR-07 to 09, NFR-02
Telemetry ingestion	Receives battery, location and fault-code readings; stores history; publishes update events	Device push	FR-06, NFR-03
Telemetry availability handler	Keeps one current telemetry snapshot per e-bike for availability searches	Broker event	FR-06, NFR-03
Freshness monitor	Every 60 s, flags e-bikes silent for 15 minutes and raises alerts	Schedule	NFR-03
Reporting and export	Nightly four-store sales report and accounting export file	Schedule	FR-10
Notification dispatcher	Sends confirmations and repair-ready notices from an outbox table	Polling	FR-15

Freshness rule *(changed, row 14)*: the availability check compares each e-bike's last-seen time with the current time directly. The 60-second monitor is for alerts only, so a bike can never stay listed past 15 minutes while waiting for the monitor to run.

Failure rules: telemetry handlers ignore duplicate events and keep only the newest reading. If the broker or telemetry service fails, e-bikes drop out of availability until they resynchronize. Modules never write to each other's tables, and telemetry stays outside the core database except for its snapshot. Notifications use the outbox pattern, so a customer is never notified about a change that was rolled back.

4.4 How the architecture meets the requirements

NFR	How it is met	Trade-off

NFR-01 Consistency	One central PostgreSQL database; Retail and Booking in one deployment	Deployment coupling
NFR-02 Concurrency	Database transactions, exclusion constraints and bike-record locks	Must be tested rigorously
NFR-03 Freshness	Separate telemetry service, broker, and a direct last-seen check	Snapshots can lag briefly
NFR-04 Access	Role checks centralized in the BFF and Identity module	All traffic must pass through the BFF
NFR-05 Performance	Redis cache-aside reads; heavy reports run off-peak	Cached data can be momentarily stale
NFR-06 Availability and recovery	Non-authoritative cache; telemetry isolated; durable broker; backups to the Section 2 recovery targets	Backup procedures must be operated
NFR-07 Usability	One BFF response per screen keeps screens simple and fast	BFF changes with every screen
NFR-08 Encryption	Column-level encryption for licence numbers and payment references; encrypted transport	Key management overhead

4.5 The case against this architecture

The strongest objection is deployment coupling. Retail and Service Booking release together, so a change to one means testing and deploying both. We kept the design anyway. The requirements put cross-store consistency and conflict-free booking first, and splitting the core into microservices now would add network calls and distributed transactions without a current need. The code-level module boundaries leave room to extract a service later if scaling demands it.

5. Data Design

5.1 How the data is stored

All business records live in one PostgreSQL database shared by the four stores. Because each record exists once, two stores cannot book the same bike or bay for the same time. Raw e-bike telemetry lives in the telemetry service's own store, so a busy feed never slows checkout. Only the latest reading per e-bike is copied into PostgreSQL for availability checks.

Store	Holds	Gains	Limits

PostgreSQL	Inventory, rentals, repairs, sales, payments, customers, users	Transactions, foreign keys, exclusion constraints that block overlapping bookings, simple four-store reports	Fixed schema
Redis	Cached search results	Keeps searches within 3 s; falls back to the database on failure	Not the source of truth; can be stale for a moment
Telemetry store	Raw readings, appended only	Telemetry never competes with store transactions	Cannot join core tables; the core uses the snapshot
Document store (rejected)	—	Flexible schema	No database rule to stop overlapping bookings, and no foreign keys

5.2 Entities

Area	Entities
Identity	User, Role, Permission, USER_ROLE, ROLE_PERMISSION
Stores and staff	Store, Mechanic, ServiceBay
Fleet and telemetry	Bike, EBike, BikeStatusHistory, BikeLocationHistory, TelemetryStatus (snapshot), TelemetryReading (in telemetry store; now includes fault codes, row 2)
Rental	Customer, RentalReservation, ReservationItem, RentalAgreement, RentalItem, RateCard, Payment, DamageReport
Workshop	RepairJob, RepairJobPart, WorkshopSchedule
Retail	Product, Part, StoreInventory, Sale, SalesItem
Messaging	NotificationOutbox

Key relationships: one reservation has one or more ReservationItems, one per bike, each with its own period and agreed rate. One rental agreement has one or more RentalItems, which record pickup, return, lateness and swaps per bike. WorkshopSchedule links a repair job, a bay and a mechanic for a time block. StoreInventory holds each item's quantity at each store.

Customer *(changed, row 16)*: holds name, contact details and an encrypted driver's licence number. The date of birth was removed because nothing uses it. A closed account is anonymized, not deleted, so rental and financial history survive.

5.3 Integrity rules

- **No double bookings:** PostgreSQL exclusion constraints on ReservationItem (per bike) and on WorkshopSchedule (per bay) reject overlapping active periods, even when two stores submit at the same moment.
- **Rental versus repair *(clarified, row 15)*:** a single constraint cannot span two tables. So the rule is enforced by locking the bike's record in one transaction while the Service Booking module checks reservations and workshop blocks, then sets In Service.
- **Agreed prices hold:** the rate is copied from RateCard onto each item at booking, so later price changes do not affect it.

5.4 Status lists *(new, row 1)*

This section is the single source for status names. Screens, services and tests use these values.

Bike status	Meaning	Rentable?
Available	Ready to rent	Yes, if the e-bike also passes FR-06
Reserved	Held for a confirmed booking	No
Rented	Out with a customer	No
In Service	In the workshop, or held after a return inspection	No
Retired	Removed from the fleet	No

Repair-job status	Meaning
Scheduled	Booked into a bay and time
In Progress	Being worked on
Ready for Pickup	Work done; customer notified
Completed	Collected, or returned to the rental fleet

5.5 How data moves

1. **Booking several bikes.** The customer picks dates, a store and bikes, and the BFF checks their role. Service Booking runs the availability search (cache, then bikes, reservation items, workshop blocks and the telemetry snapshot) and reads the current rate. In one transaction it locks the bikes, writes the reservation and one item per bike, and writes an outbox row; the exclusion constraint rejects any overlap. The deposit goes through the gateway, and the reservation is confirmed, or released if payment fails. After the commit the cache is cleared and the confirmation is sent.
2. **Closing a rental.** The agent scans the returned bike and records its condition and return store. In one transaction the system sets the return time, calculates any overage from the stored rate, and closes the agreement when every bike is back. It records any damage, the move to the return store, and the new status (Available, or In Service with a repair job). The balance is charged, and after the commit the cache is cleared so all stores see the bike as available again.
3. **An e-bike reading.** The telemetry service validates and stores the reading, then publishes an event. The core ignores duplicates, updates the snapshot only if the reading is newer, and refreshes the cache. Availability searches read only the snapshot and exclude any e-bike whose last-seen time is over 15 minutes ago.

5.6 Checking the model against real cases

Before asking an AI tool for a model, we wrote ten edge cases the data must handle (E1–E10). These included a booking for several bikes, a bike moved between stores, a price change mid-rental, an account closure, one bike returned late, a mid-rental swap, and a refund after a deposit. The generated model looked complete but failed most of them, and its cascade delete would have erased rental history. Testing it also exposed gaps in our own Week 5 model. Those findings produced ReservationItem, RentalItem, RateCard, BikeLocationHistory, reservation-level payments and customer anonymization.

6. Quality and Test Plan

6.1 Quality processes

Quality is built into every weekly iteration, not inspected at the end. Quality assurance has two parts: preventive practices that make defects less likely, and testing that shows how the system actually behaves.

Practice	How we apply it	Status

Acceptance-criteria review	Every story needs a criterion with a number (3 s, 15 min) before work starts	In use
Edge-case analysis	Concurrent bookings, swaps, late returns and stale telemetry are listed before building	In use (Week 6)
Independent review	Someone other than the author reviews each change within 24 hours; the author never verifies their own work	Team policy
Traceability	Each requirement links to its tests; unsupported requirements are marked as such	Section 6.3
Blameless review	Escaped defects are discussed at the Wednesday sync, modelled on SRE postmortems	Planned
Automated checks and regression	Build, lint, schema and regression checks on every merge	Planned: no automated tests exist yet

Definition of done: every acceptance criterion is checked and its evidence recorded; a second teammate has reviewed the change; defects are fixed or logged with severity and an owner; requirements, tests and traceability are consistent; and anything the build cannot satisfy is reported as failed or unsupported, never passed.

6.2 Testing inside our process

When	Who	Testing work
Sprint planning	Deliverable owner and team	Make criteria verifiable; identify boundary cases, test data and dependencies
During build	Developer	Unit checks of validation, status changes and boundaries
Once integrated	A peer tester who did not build it	Functional and integration checks against the criteria; reproducible failures recorded
End of sprint	Team	Review passes and failures; decide whether defects block the story

Agile makes a late test phase awkward. We compensate with stable criteria each sprint, early integration testing, and a small regression set. Concurrent-user tests need a scheduled run with a shared clock and dataset.

6.3 Key tests

Test	What it checks	Requirements	Status against the MVP
GB-INT-01	A committed bike-record update reads the same from every store filter	FR-02 (partial), NFR-01	Bike records only; product quantities unsupported
GB-CON-02	Two simultaneous bookings for one bike: one returns 201, the other 409; exactly one item committed	FR-04, FR-07, NFR-02	Reservations testable; bay scheduling unsupported
GB-FUNC-03	An e-bike silent for 20 minutes cannot be reserved; one seen 10 minutes ago can	FR-03, FR-06, NFR-03	Reservation guard testable; availability search unsupported
GB-SEC-04	Unauthenticated requests to repair endpoints are refused	FR-11, FR-12, NFR-04	Expected to fail: no login yet (High-severity defect)
GB-USA-05	9 of 10 untrained Rental Agents complete a rental unaided in under 3 minutes	FR-03, FR-04, NFR-07	Planned; not yet run
GB-INT-06 *(new, row 15)*	A bike set to In Service by a workshop job cannot be reserved for an overlapping period, even when both requests race	FR-03, FR-07, NFR-02	Blocked until workshop scheduling exists

These tests are specified, not yet passed.

6.4 Where we stop, and the risk we accept

We test by risk. Effort goes where a failure is both likely and costly.

Tier	Requirements	Depth
1: Deep	NFR-01, NFR-02, NFR-03	Several tests each, including boundaries (14:59 and 15:01 for telemetry) and concurrency
2: Targeted	NFR-04, NFR-08, NFR-05	One test per criterion
3: Light	NFR-06, NFR-07	A cache-failure check and the GB-USA-05 protocol

Not tested: full recovery and uptime (no hosted environment), seasonal peak load, usability beyond rentals, wide browser coverage, every rate combination, the live payment gateway and physical e-bikes (sandbox and simulated only), and penetration testing. Each has a trigger that

reopens it, such as a hosted deployment or a pricing defect reaching a customer. A second kind of residual risk is what the MVP does not yet do: login and roles, availability search by period, the parts-inventory API and workshop scheduling.

6.5 When a defect escapes

Anyone who finds a defect records the store, workflow, expected and actual behaviour, reproduction steps and customer impact. A coordinator manages the response, the lead engineer assigns a fix owner, and a teammate who did not write the fix verifies it.

Severity	Example	Response
Critical	Customer data exposed; a known-unsafe bike released	Notify coordinator, lead and business owner immediately; contain first
High	Double booking, wrong charge, a core workflow lost	Investigate immediately; notify affected stores
Moderate	Local failure with a safe workaround	Fix within the iteration
Low	Cosmetic issue	Backlog

Containment comes first: remove a bike from availability, restrict a function, or roll back (Part 3). A fix closes only when the original failure is reproduced, an independent re-test passes, and the affected records are reconciled. For example, a return fix must show that a bike placed In Service stays unavailable.

Part 2: Where the Components Disagreed, and What We Did

We read the six components side by side and found 18 disagreements. Each row records what we found, what we changed, which document changed, and where the change now appears in this package. Rows 1–3 are presented on Slide 8. Where a component disagreed with the MVP built in Week 7, the agreed requirements were the authority, unless the team decided the MVP behavior was the better design.

#	What we found	What we changed	Document changed	In this package

1	**Repair status named differently.**Every component agreed that a bike under repair cannot be rented. But Weeks 3 and 5 call it "In Service", Week 4 calls it "Maintenance", and Screen 1 shows "Active". Repair stages also differ between Week 3 FR-08 and the Week 7 MVP (READY_FOR_PICKUP). Week 7 could not test the rule because the MVP cannot create workshop schedules.	Made the data design the single source of two status lists, one for bikes and one for repair jobs. The screens and tests now use those names.	Week 4, Week 6, Week 7	5.4, 3.1–3.2, 6.3
2	**A Week 4 screen with no data behind it in Week 6.** Journey 2 sends an unlock command to a smart lock, and Journey 3 reads motor fault codes. But Week 5 telemetry only receives readings from the bike (battery, location), Week 6 stores no fault codes, and Week 3 puts hardware out of scope.	Removed the lock command. Added fault codes to TelemetryReading and FR-06, so the safety check in Journey 3 has data behind it.	Week 3, Week 4, Week 6	2.1 FR-06, 3.1, 5.2
3	**One bike or several per booking.**Week 3 FR-04/05 and Week 4 Screen 3 allow one bike, while the Week 6 ReservationItem and RentalItem, and the Week 7 MVP, support several.	Rewrote FR-04/05 to allow one or more bikes, because families and groups rent together. Screen 3 now allows selecting several bikes.	Week 3, Week 4	2.1, 3.2
4	**Payment scope.** Week 3 NFR-08 makes payment encryption conditional, while the Week 3 Decision Record says all payment data is protected. Weeks 4–6 take deposits, overage charges and refunds through a gateway.	Brought payments into scope through an external gateway that holds the card data. Added FR-13 and made NFR-08 unconditional.	Week 2, Week 3	1.3, 2.1–2.2

5	**Message broker.** Week 5 chose RabbitMQ and rejected Kafka as too much operating overhead, but the Week 7 MVP runs on Kafka.	Kafka, because it is the broker the MVP runs	Week 5	4.1
6	**Roles in the journeys.** Week 4 has a Store Clerk processing rentals and a "Shop Technician" handling returns, while the Week 3 access rules give these jobs to Rental Agents and Mechanics.	Renamed the personas to match the access rules.	Week 4	3.1
7	**What "inventory" means.** Week 3 means product and part quantities, while Week 4 Screen 1 and Week 7 GB-INT-01 mean the bike fleet.	Renamed Screen 1 to Fleet. Recorded the parts-inventory screen and API as not yet built.	Week 4, Week 7	3.2, 6.3
8	**Requirements with no screen.** No screens exist for FR-05, FR-07, FR-09, FR-10 or FR-11.	Recorded these as interface gaps, with the customer screens first.	Week 4	3.2
9	**NFR numbering.** Week 5 renumbered the NFRs and dropped usability, and Week 6 Table 3 omitted three NFRs.	Week 3 numbering is used throughout; usability is now included.	Week 5, Week 6	4.4
10	**Performance targets.** The Week 3 AI Use Record says 2 s / 3 s, while NFR-05 and Weeks 5–7 use 3 s / 5 s.	Kept 3 s / 5 s and corrected the AI Use Record text.	Week 3	2.2
11	**Recovery targets.** Week 5 left recovery targets undefined, and no component set a data-loss limit.	Cited the 30-minute restore time and added a data-loss limit of 5 minutes.	Week 3, Week 5	2.2, 4.4

12	**Seasonal peak load.** Week 2 relied on auto-scaling, but Week 3 has no scalability requirement and Week 7 does not test peaks.	Kept the managed platform as a constraint, and recorded peak load as an accepted, untested risk.	Week 2, Week 7	1.4, 6.4
13	**Battery threshold.** "Meets rental conditions" and "below threshold" appear, but no component gives a number.	Set a minimum charge of 30% in FR-06.	Week 3	2.1
14	**Freshness timing.** The 60-second monitor could leave a silent e-bike listed for about 16 minutes, against a 15-minute rule.	Availability compares the last-seen time directly; the monitor only raises alerts.	Week 5	4.3
15	**Rental versus repair overlap.** Week 6 notes a constraint cannot span two tables, but Week 7 says the database enforces the rule.	Documented the bike-record lock that enforces it, and added test GB-INT-06.	Week 6, Week 7	5.3, 6.3
16	**Designed with no requirement.**Pricing, notifications and a customer date of birth have no requirement behind them.	Added FR-14 and FR-15, and removed date of birth.	Week 3, Week 6	2.1, 5.2
17	**MVP missing specified features.**The MVP has no login, no availability search by period, no parts API and no workshop scheduling.	Kept the requirements and recorded the gaps. These drive the conclusion.	Week 7	6.4, Conclusion
18	**Ownership rotation.** Weeks 2–4 say it rotates weekly; Week 7 says every two weeks.	Used the weekly rotation. Charter roles (Coordinator, Lead Engineer, Documentation Lead, Review Lead) still rotate every two weeks under the Week 1	Week 7	1.5

		charter; Week 7 was describing those roles.		

Part 3: Deployment and Maintenance Outlook

How the system would be released

We would release one store at a time, in three steps.

1. **Prepare.** Back up existing store data, configure the system on the managed cloud platform, and verify the release in a test environment. The core application and the telemetry service are separate deployables, so each is released on its own.
2. **Pilot and validate.** Go live at one store in the off-season, never before a local event weekend, when rental demand peaks. Validate inventory updates, rentals, returns, repairs and user access against the Section 6 tests.
3. **Roll out gradually.** Train staff, then add the other three stores once the pilot has run cleanly for two weeks.

Every release, including later updates, follows the same checklist so it never depends on one person remembering the steps. Before it: a verified backup, the release version, the environment configuration and credentials, and a pass of the Section 6 tests. During it: database migrations first, then the core application, then the telemetry service. After it: a teammate who did not build the release runs a short production check against expected results (a fleet search, a two-bike booking, a conflicting booking that must be rejected, a return, and a stale e-bike that must be unavailable), the coordinator confirms with the store manager, and anything that did not go to plan is added to the checklist.

Every release is a tagged version in version control, together with its migrations and configuration. Only changes that passed the independent review in Section 6.1 are tagged for release. We record which version each store runs, so a reported fault can be traced to an exact build and the rollback below always has a known version to return to.

Database changes are applied as migrations that only add tables and columns, never remove them, so the previous application version still runs against the new schema. A column being retired, such as the customer date of birth (Part 2, row 16), stops being used in one release and is removed in a later one, once no running version reads it.

Rollback plan

- **Application fault:** switch back to the last stable release, which we keep available. This takes minutes, well inside the 30-minute recovery target (NFR-06).
- **Data fault:** restore validated data from backup, within the data-loss limit in NFR-06. Before restoring, we check whether the fault can be corrected forward with a new

migration, because a restore loses bookings and deposits taken since the backup. After any restore, rentals are reconciled against the payment gateway's records.

- **Telemetry fault:** the system fails safely. E-bikes drop out of availability, and standard bikes keep renting.
- **Manual fallback:** the pilot store keeps its old spreadsheets read-only for two weeks.

What will be hardest to maintain, and why

Booking integrity will be hardest to maintain. The guarantee that no bike or bay is ever double-booked is not held in one place. It depends on three mechanisms working together: database exclusion constraints, the bike-record lock inside a transaction, and clearing cached availability after every commit (Sections 4.2 and 5.3). Any schema change can quietly break one of them, just as the multi-bike bookings added in Week 6 moved the overlap rule to a new table. The MVP has no automated tests yet, so a regression could reach a store before anyone notices.

The second-hardest is the deployment coupling we accepted in Section 4.5. A change to Retail means redeploying Service Booking too, which is riskier during peak rental season.

Maintenance will be most of this system's lifetime cost, and the owners asked for that cost to be predictable (Section 1.2). We expect four kinds of work: corrective (escaped defects, handled through Section 6.5); adaptive (changes forced by the payment gateway, the email and SMS provider, or the e-bike telemetry feed); perfective (the screens not yet designed in Section 3.2, and new rates or notices under FR-14 and FR-15); and preventive (the automated concurrency and telemetry tests recommended in the conclusion, which protect booking integrity on every change).

Conclusion: What We Would Build First, and Why

We recommend funding the next increment as **role-based login plus availability search by requested period, protected by automated concurrency and telemetry tests**.

The current MVP already proves the hardest part of the design. The shared fleet records work across stores, the database blocks overlapping bookings, and stale e-bikes are refused at reservation time. It cannot yet go in front of staff or customers, for two reasons:

1. **There is no login or role-based access** (FR-12, NFR-04). Any request can read or change repair records today, so the system cannot hold real customer data. This is our only High-severity known defect, and it would become Critical in production.
2. **There is no availability search by period** (FR-03). A Rental Agent cannot see which bikes are free for the dates a customer asks for, which is the core rental task. Without it, the performance target (NFR-05) cannot be measured and the usability test (GB-USA-05) cannot fully pass.

We would build automated tests in the same increment, not after it. Part 3 shows that booking integrity is the hardest thing to maintain, so the concurrency test (GB-CON-02, repeated to the 100-scenario criterion) and the telemetry boundary tests (14:59 and 15:01) run on every change from the start.

Workshop scheduling, the parts-inventory API and the customer screens follow in the next increment. They depend on login being in place, and they unlock the remaining tests (GB-INT-06 and the bay half of GB-CON-02).